3X AI TESTER BLUEPRINT

Python Notes

All Basic Concepts — Chapter 11: Python Learning

Compiled from the Eraser workspace: **3x AI Tester Blueprint**

Course resources: python.org • sdetclub.com • thetestingacademy.com

Table of Contents

1. Python Learning Roadmap
2. Introduction to Python
3. Syntax Rules, Comments, Keywords & Identifiers
4. Variables & Data Types
5. Literals
6. User Input & Type Conversion
7. Operators
8. Strings
9. Conditional Statements (if / elif / else)
10. Loops (for, while, break, continue, pass)
11. Switch / Match Statement
12. Functions (Built-in, User-defined, Scope, Decorators, Lambda)
13. Collections: List, Tuple, Set, Dictionary
14. Map, Filter & Lambda
15. OOPs — Class, Object, Constructor
16. OOPs — Encapsulation, Inheritance, MRO
17. OOPs — Polymorphism, Abstraction, Static Members
18. Exception Handling
19. Modules, Packages & PIP
20. Virtual Environment (venv)
21. OS Module (Operating System Interactions)
22. File IO & .env Config Files
23. PyTest Basics for Test Automation
24. Quick Revision Cheat Sheet

1. Python Learning Roadmap

Goal of this module (from the course blueprint): build the Python foundation needed for AI-agent frameworks used later in the course — CrewAI, LangChain, LangGraph, LangSmith, FastMCP, DeepEval and test automation with PyTest + Playwright.

Learning sequence

- **Basics** — Identifiers, Keywords, Operators, Literals and Variables.
- **Control flow & logic building** — if, for loop, switch (match), while, do-while pattern.
- **Core building blocks** — Arrays (Lists), Functions, Strings, Exception Handling and misc topics.
- **Mixed concepts** — Import-Export, using utilities, libraries and many more.
- **OOPs concepts** — followed by library imports and building simple products.

Practice resources mentioned in the course

- Install: `www.python.org/downloads/`
- Tasks: `www.sdetclub.com/c/ai-tester-3x-only/` (Task 1 — Python program & coding questions)
- Recordings: `courses.thetestingacademy.com/`

> **Tip:** Type every example yourself in VS Code with the Python extension + Code Runner. Reading code is not the same as writing code.

2. Introduction to Python

What is Python?

- Developed in 1990 (by Guido van Rossum; first public release in 1991).
- A **high-level programming language** — close to human language, human-friendly and readable.
- Widely used for web frameworks (Flask, Django), data tools (Pandas, NumPy), AI libraries (Transformers, LangChain), testing tools (PyTest, Playwright) and automation (Requests, Selenium).

Compiled or Interpreted?

> **Answer: Both.** Python source is first compiled to bytecode (a `.pyc` file) and then the bytecode is interpreted line by line by the Python Virtual Machine.

```
source.py  --compile-->  source.pyc (bytecode)  --interpret line by line-->  execution
```

Source code

- Source code is the code / syntax that **you (a human) write** following the rules of the high-level language.
- It is **human-understandable** plain text.

Where should we write the code? (IDEs)

An **IDE (Integrated Development Environment)** takes care of your files, logs, data, Python environments and more — "acts like a mother" for your project.

Level	Tool	Notes
Beginner	VS Code + Code Runner (+ GitHub Copilot / Claude Code)	Light, free, recommended setup for this course
Medium	PyCharm Community Edition	Full Python IDE, free
Advanced	PyCharm Professional	Paid, web frameworks & DB tools

> **Why not Notepad?** Writing code in plain Notepad is not practical — managing 15,000 test cases across Python files without an IDE's navigation, debugging and refactoring tools is impossible.

Installation

- Download from `www.python.org/downloads/`
- Mac → `.dmg` | Windows → `.exe` | Linux → `.sh` / `.deb`
- Verify from a terminal: `python --version` (or `python3 --version` on Mac/Linux)

```python
# Lab001 — your first program
print("Hello, World!")   # output: Hello, World!
```

3. Syntax Rules, Comments, Keywords & Identifiers

Python syntax (rules)

- Statements end with a newline — no semicolons needed.
- **Indentation defines blocks** — use 4 spaces or 1 Tab consistently. Wrong indentation = `IndentationError`.
- Python is **case-sensitive**: `print()` works, `Print()` fails.

Comments

- A comment is code that will **not be executed** — used for explanation.
- Single line: `# This is a comment`
- Multi-line: triple-quoted strings `""" ... """` used as documentation.

```
# This is a single-line comment - it will not be executed
"""
This is a
multi-line comment / docstring
"""
print("comments demo")
```

Formatting tip: in VS Code use the default formatter (Black) — together with GitHub Copilot / Claude Code it keeps spacing and style consistent.

Keywords (Reserved Words)

- Keywords are also called **reserved words**.
- We **cannot** use a keyword as a variable name, function name or any other identifier.
- Keywords are **case-sensitive** (all Python keywords are lowercase, except `True`, `False`, `None`).

```
>>> import keyword
>>> print(keyword.kwlist)
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class',
 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global',
 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise',
 'return', 'try', 'while', 'with', 'yield']
>>> print(keyword.iskeyword("class"))   # True
```

Identifiers

- An identifier is the **name of a variable**, function, class or object.
- Rules:
  - Only letters, digits and underscore `_` (alphanumeric is possible as a name).

- **Cannot start with a digit** — do not have a number as the first thing in a name.
- `_` alone can be a valid variable name.
- Identifiers are case-sensitive: `age`, `Age`, `AGE` are three different names.

```
name = "Pramod"      # valid
_ = 99               # valid
my_10th_marks = 89   # valid (alphanumeric + underscore)
10th_marks = 89      # INVALID - cannot start with a digit
class = "LCFE"      # INVALID - keyword used as identifier
```

4. Variables & Data Types

Creating a variable (syntax)

A variable is a **container** with three parts:

Part	Meaning	Example
Data type of the container	What kind of value it holds	`int`
Name of the container	Variable name (identifier)	`age`
Value stored in the container	The actual data	`65`

```
age = 65        # no type declaration needed - Python infers it
age = "sixty"  # allowed - variables can be reassigned to another type
print(type(age))  # <class 'str'>
```

Python data types

Numeric types

- **int** — whole numbers, e.g. `-1, -2, -4, 0, 13, 1234, 98999, 9876543210`
- **float** — decimals, e.g. `3.14, 18.45, 3.12345678`
- **bool** (Boolean) — `True` or `False`, e.g. `is_married = True`

String type

- **str** — a bunch of characters: P,R,A,M,O,D → `"Pramod"` (covered in detail in Chapter 8).

List type

- **list** `[ ]` — the Python "array": ordered, changeable, allows duplicates and mixed types.

```
shopping_list = ["milk", "bread", "butter", "panner"]
my_10th_marks  = [100, 91, 93, 56, 89]
my_mixed       = ["dasda", 123]          # mixed types allowed
```

Advanced data types

- **set** `{ }`, **dict** `{k: v}`, **tuple** `( )`, **frozen set**, **binary / bytes** — covered in Chapter 13.

Check any value's type

```
print(type(3))          # <class 'int'>
print(type(3.14))       # <class 'float'>
print(type("hi"))     # <class 'str'>
print(type([1,2]))      # <class 'list'>
print(type(True))     # <class 'bool'>
```

5. Literals

A **literal** is the raw value written directly in the source code.

Literal category	Examples
Numeric	`int` (`10` , `0x1A`), `float` (`3.14`), `bool` (`True` , `False`)
String	`'single'` , `"double"` , `"""triple"""`
NoneType	`None` — represents "no value"
Ellipsis	`...` — placeholder, also used in type hints
Collection	`[1, 2]` (list), `(1, 2)` (tuple), `{1, 2}` (set), `{'a': 1}` (dict)

```
a = 10            # int literal
b = 3.14          # float literal
c = True          # bool literal
d = "Pramod"    # string literal
e = None          # None literal
f = ...           # Ellipsis literal
```

Escape characters in strings

```
print("Line1\nLine2")        # \n new line
print("He said \t\"hi\"")    # \t tab, \" quote inside quotes
print(r"C:\new\folder")     # raw string - backslash is literal
```

6. User Input & Type Conversion

Taking user input

- `input()` takes "the string that you want to ask from the user" as its prompt.
- **It always returns a string** — convert (cast) it when you need a number.

```
name = input("What is your name? ")
age  = int(input("What is your age? "))    # cast string -> int
print(type(name), type(age))
print(f"Hello {name}, next year you will be {age + 1}")
```

Type conversion (casting)

Function	Converts to	Example
`int(x)`	integer	`int("42")` → 42
`float(x)`	float	`float("3.14")` → 3.14
`str(x)`	string	`str(99)` → "99"
`bool(x)`	boolean	`bool("")` → False, `bool("a")` → True
`list(x)`	list	`list("abc")` → ['a','b','c']

Interview Q: What does `input()` return? — Always a **string**, even if the user types a number. Use `int()` / `float()` to convert.

```
# Lab022 - sum of two numbers from the user
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
print(f"Sum = {a + b}")
```

7. Operators

Arithmetic operators

```
print(10 + 3)   # 13   addition
print(10 - 3)   # 7    subtraction
print(10 * 3)   # 30   multiplication
print(10 / 3)   # 3.33 division - always returns float
print(10 // 3)  # 3    floor division - drops the decimals
print(10 % 3)   # 1    modulus - remainder
print(10 ** 3)  # 1000 exponent - power
```

Comparison operators (return True / False)

```
print(10 == 10)   # True   equal
print(10 != 10)   # False  not equal
print(10 > 3)     # True   greater than
print(10 < 3)     # False  less than
print(10 >= 10)   # True   greater or equal
print(10 <= 3)   # False  less or equal
```

Logical operators

```
print(True and False)   # False - both must be True
print(True or False)    # True  - at least one True
print(not True)        # False - inverts

age = 25
if age >= 18 and age <= 60:
    print("Eligible")
```

Assignment operators

```
x = 5; x += 2   # x = 7
x -= 2; x *= 2; x /= 2; x //= 2; x %= 2; x **= 2
```

Membership and identity operators

```
langs = ["python", "java"]
print("python" in langs)      # True  - membership test
print("go" not in langs)   # True

a = [1, 2]; b = [1, 2]
print(a == b)   # True  - same VALUES
print(a is b)    # False - different objects in memory
```

Bitwise operators

```
print(5 & 3)   # 1   AND
print(5 | 3)    # 7   OR
print(5 ^ 3)    # 6   XOR
print(~5)       # -6  NOT
print(5 << 1)   # 10  left shift
print(5 >> 1)   # 2   right shift
```

Ternary (conditional) operator

```
age = 20
status = "Adult" if age >= 18 else "Minor"
print(status)   # Adult
```

8. Strings

Creating strings

```
s1 = 'single quotes'
s2 = "double quotes"
s3 = """triple quotes -
can span multiple lines"""
```

Indexing and slicing

```
s = "Python"
print(s[0])     # 'P'   - index starts at 0
print(s[-1])    # 'n'   - negative index counts from the end
print(s[0:3])   # 'Pyt' - slice (start:stop)
print(s[::-1])  # 'nohtyP' - reverse a string (interview favourite)
print(len(s))   # 6
```

Common string methods

Method	What it does
`upper()` / `lower()`	Convert case
`strip()`	Remove leading/trailing spaces
`replace(a, b)`	Replace substring a with b
`split(sep)`	Split into a list
`join(list)`	Join list into one string
`find(sub)` / `index(sub)`	Position of substring (-1 vs error)
`startswith()` / `endswith()`	Check prefix / suffix
`count(sub)`	Number of occurrences
`isdigit()`, `isalpha()`	Character class checks
`capitalize()`, `title()`	Case formatting

```
s = "  ai tester blueprint  "
print(s.strip().title())        # 'Ai Tester Blueprint'
print("-".join(["ai", "tester"]))  # 'ai-tester'
print("a,b,c".split(","))      # ['a', 'b', 'c']
```

String formatting (f-strings)

```
name, score = "Pramod", 98.765
print(f"Hello {name}")            # Hello Pramod
print(f"Score: {score:.1f}")        # Score: 98.8
print(f"{name=}")                 # name='Pramod' (debug)
```

Remember: strings are **immutable** — every method returns a new string; the original never changes in place.

9. Conditional Statements (if / elif / else)

Simple if

```
age = 20
if age >= 18:
    print("You can vote")   # note the 4-space indentation of the block
```

if / else

```
num = 7
if num % 2 == 0:
    print("Even")
else:
    print("Odd")          # Odd
```

if / elif / else (multiple conditions)

```
marks = 82
if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"       # this branch runs
elif marks >= 60:
    grade = "C"
else:
    grade = "F"
print(grade)   # B
```

Nested if and short-hand

```
if age > 18:
    if age < 60:
        print("Eligible")

result = "Pass" if marks >= 35 else "Fail"   # one-liner (ternary)
```

Logic-building practice: write small programs — largest of 3 numbers, leap year, vowel checker, grade calculator — the course tasks focus on exactly this kind of logic building.

10. Loops (for, while, break, continue, pass)

for loop — iterate over a sequence

```
for i in range(5):          # 0 1 2 3 4  (stop value excluded)
    print(i, end=" ")

for i in range(1, 11, 2):    # 1 3 5 7 9  (start, stop, step)
    print(i, end=" ")

for item in ["milk", "bread"]:
    print(item)

for idx, val in enumerate(["a", "b"]):   # index + value together
    print(idx, val)
```

while loop — run while condition is True

```
i = 1
while i <= 5:
    print(i, end=" ")   # 1 2 3 4 5
    i += 1               # ALWAYS update the counter or the loop is infinite
```

do-while behaviour in Python

Python has no `do-while`. Emulate it with `while True` + `break`:

```
while True:
    cmd = input("Command: ")
    if cmd == "quit":
        break            # exit the loop
    print(f"Running {cmd}")
```

break, continue, pass

```
for i in range(1, 10):
    if i == 5:
        break          # stop the entire loop at 5
    if i % 2 == 0:
        continue     # skip even numbers, go to next iteration
    print(i, end=" ")   # 1 3

for i in range(3):
    pass           # do nothing - placeholder for future code
```

Common IQ pattern: print a right-angled triangle of stars with nested loops — outer loop for rows, inner loop for columns.

```
for row in range(1, 6):
    print("*" * row)
# *
# **
# ***
# ****
# *****
```

11. Switch / Match Statement

Python 3.10+ replaces the classic `switch` of other languages with **structural pattern matching** — `match` / `case`.

```
def http_status(code):
    match code:
        case 200:
            return "OK"
        case 404:
            return "Not Found"
        case 500:
            return "Server Error"
        case _:                    # default case
            return "Unknown"

print(http_status(404))   # Not Found
```

Matching multiple values and guards

```
match browser:
    case "chrome" | "edge":      # multiple values with |
        print("Chromium based")
    case "firefox":
        print("Gecko based")

match score:
    case s if s >= 90:              # guard condition
        print("Excellent")
```

12. Functions

A function is a reusable, named block of code: `def sum(): ...` then call `sum(3, 4)`. Functions are the primary tool for dividing a program into logical pieces.

Types of functions in Python

- **Built-in** — `print()`, `len()`, `input()`, `type()`, `range()`, `id()`, `sum()`, `max()`, `min()`, `sorted()` ...
- **User-defined** — the ones you write with `def`.

User-defined function patterns (as taught in the course)

1. No return type

```
def greet():
    print("Hello!")     # returns None implicitly

greet()
```

2. Return type

```
def add(a, b):
    return a + b        # gives the value back to the caller

result = add(3, 4)     # 7
```

3. Default parameter

```
def greet(name="Guest"):
    print(f"Hello {name}")

greet()          # Hello Guest
greet("Pramod")   # Hello Pramod
```

4. Keyword arguments

```
def profile(name, role):
    print(f"{name} - {role}")

profile(role="SDET", name="Ashu")   # order does not matter
```

5. Multiple parameters, *args and **kwargs

```
def total(a, b, c):            # multiple fixed params
    return a + b + c

def add_all(*args):            # any number of positional args - tuple
    return sum(args)
print(add_all(1, 2, 3, 4))    # 10

def info(**kwargs):            # any number of keyword args - dict
    for k, v in kwargs.items():
        print(k, "=", v)
info(name="AWS", tool="Selenium")
```

> **Function within a function is allowed in Python** (nested / inner functions) — this is exactly what makes decorators possible.

```
def outer():
    def inner():
        print("inner logic")
    inner()                # call inner from outer's body
outer()
```

Scope of variables (local vs global)

```
total = 0                  # global scope
def calc():
    local_total = 10       # local scope - exists only inside calc()
    global total
    total += local_total   # 'global' keyword lets you modify the global
calc()
print(total)               # 10
```

LEGB rule: Python looks up names in order — **L**ocal → **E**nclosing → **G**lobal → **B**uilt-in.

Lambda expressions (anonymous functions)

```
square = lambda x: x ** 2
print(square(5))                    # 25
add = lambda a, b: a + b
print(add(3, 4))                    # 7
```

Decorators

A decorator is a function that takes another function, adds behaviour around it, and returns the enhanced function — applied with `@name`.

```python
def logger(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        result = func(*args, **kwargs)
        print("Done")
        return result
    return wrapper

@logger
def create_bug(summary):
    print(f"BUG created: {summary}")

create_bug("Login fails on Chrome")
# Calling create_bug
# BUG created: Login fails on Chrome
# Done
```

13. Collections: List, Tuple, Set, Dictionary

The four core collection types (remember the brackets): **tuple** `( )`, **list** `[ ]`, **set/dict** `{ }`.

Type	Syntax	Ordered	Mutable	Duplicates	Indexed
List	`[1, 2, 3]`	Yes	Yes	Allowed	Yes
Tuple	`(1, 2, 3)`	Yes	No	Allowed	Yes
Set	`{1, 2, 3}`	No	Yes	Not allowed	No
Dict	`{1: "a"}`	Yes (3.7+)	Yes	Keys unique	By key

List ("array") + summary of list methods

```
shopping_list = ["milk", "bread", "butter"]
shopping_list[0] = "curd"       # change item by index
print(shopping_list[-1])       # 'butter' - negative indexing
print(shopping_list[0:2])      # ['curd', 'bread'] - slicing
```

Method	What it does
`append(x)`	Add one item at the end
`extend(iter)`	Add all items of another list/iterable
`insert(i, x)`	Insert item x at position i
`remove(x)`	Remove first occurrence of value x
`pop(i)`	Remove & return item at index i (default: last)
`clear()`	Empty the whole list
`index(x)`	Position of the first occurrence of x
`count(x)`	How many times x appears
`sort()`	Sort the list in place
`reverse()`	Reverse the list in place
`copy()`	Shallow copy of the list

```
nums = [5, 2, 9, 2]
nums.append(7); nums.sort(); print(nums)    # [2, 2, 5, 7, 9]
print(nums.count(2))                         # 2
last = nums.pop(); print(last, nums)         # 9 [2, 2, 5, 7]
```

Tuple

```
marks = (100, 91, 93, 56, 89)
print(marks[1])          # 91
print(len(marks))         # 5
# marks[1] = 95 - TypeError! tuples are immutable
single = (42,)           # single-element tuple NEEDS the comma
```

Set

- A set is an **unordered** collection that is iterable, **mutable**, and has **no duplicate elements**.
- It does not allow duplicates and is mainly used for **membership testing, removing duplicates and mathematical set operations**.
- Collection of unique items; no order, unindexed.

```
tags = {"smoke", "regression", "smoke"}
print(tags)                         # {'smoke', 'regression'} - duplicate removed
print("smoke" in tags)         # True - fast membership test
print(set([1, 2, 2, 3]))            # {1, 2, 3} - de-duplicate a list

a = {1, 2, 3}; b = {3, 4, 5}
print(a | b)     # {1,2,3,4,5}  union
print(a & b)     # {3}          intersection
print(a - b)     # {1,2}        difference
print(a ^ b)     # {1,2,4,5}    symmetric difference
```

Dictionary (dict / map)

```
tester = {"name": "Ashu", "role": "SDET", "exp": 5}
print(tester["role"])               # SDET
tester["exp"] = 6                     # update a value
tester["suite"] = "regression"      # add a new key
print(tester.get("email"))          # None - safe access, no KeyError
print(tester.get("email", "N/A"))   # N/A - with default
tester.pop("suite")                  # remove a key

for k, v in tester.items():         # loop over key-value pairs
    print(k, "=", v)
print(tester.keys(), tester.values())
```

14. Map, Filter & Lambda

`map()` and `filter()` are built-in higher-order functions that are usually combined with `lambda`. Both return iterators — wrap them in `list()` to see results.

map() — transform every item

```
nums = [1, 2, 3, 4]
squares = list(map(lambda x: x ** 2, nums))
print(squares)    # [1, 4, 9, 16]

names = ["pramod", "ashu"]
print(list(map(str.upper, names)))   # ['PRAMOD', 'ASHU']
```

filter() — keep items that pass a test

```
nums = [10, 25, 30, 45]
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens)      # [10, 30]
```

map vs filter — the difference

	map()	filter()
Purpose	Transforms each element	Selects elements by condition
Output length	Same as input	Same or fewer items
Function returns	New value	True / False

Comprehensions (the Pythonic alternative)

```
squares = [x ** 2 for x in range(1, 5)]        # [1, 4, 9, 16]
evens   = [x for x in nums if x % 2 == 0]     # [10, 30]
lookup  = {k: v for k, v in [("a", 1)]}         # dict comprehension
```

15. OOPs Part 1 — Class, Object, Constructor

Why OOP?

Procedural (C)	Object-Oriented (Python, Java)
Emphasis on *doing things* — the program is divided into functions	Emphasis on *data* — the program is divided into classes and objects
Data and functions are separate	Data (attributes) and functions (methods) are bound together
Productivity is lower	Productivity is high — reuse via classes

The four pillars + the building blocks

- **Object • Classes • Encapsulation • Abstraction • Inheritance • Polymorphism**
- Everything you create in a program can be divided into functions — but OOP organises data members (variables) and methods (functions) inside classes.

Class — the blueprint

- A **class** is a user-defined data type which defines its properties and its methods.
- A class contains **attributes** (properties / data variables) and **behaviors** (methods / member functions).

Object — the instance

- An **object** is a run-time entity — the instance of a class, a real-world entity created from the class blueprint.
- All data members and member functions of the class are accessed through objects.

```
class TestCase:                  # class = blueprint
    def __init__(self, tc_id, module):
        self.tc_id = tc_id           # attribute / data variable
        self.module = module

    def show(self):                # method / member function
        print(f"{self.tc_id} - {self.module}")

tc1 = TestCase("TC01", "Login")   # object = instance of the class
tc1.show()                        # TC01 - Login
```

Constructor — __init__ and self

- A constructor is a **special type of function that is called automatically** when you create an object.
- In Python its name is always `__init__`, and `self` refers to the **current object**.
- A constructor **does not return anything** (no `return` statement).

- Two types:
  - **Default constructor (DC)** — takes no arguments.
  - **Parameterised constructor (PC)** — takes arguments.

```
class Bug:
    def __init__(self):                  # default constructor
        self.status = "NEW"

    def set_priority(self, p):           # regular method with param
        self.priority = p

b = Bug()           # __init__ runs automatically here
print(b.status)     # NEW
```

Instance variables vs class (static) variables

```
class Suite:
    framework = "PyTest"           # CLASS variable - shared by all objects

    def __init__(self, name):
        self.name = name             # INSTANCE variable - unique per object

print(Suite.framework)   # access class variable via class name
```

16. OOPs Part 2 — Encapsulation, Inheritance, MRO

Encapsulation

- **Bind the data variables with the methods** — wrapping or binding the instance variables / data variables with the methods is encapsulation.
- Instance variables can be accessed only through the class's methods.
- A method is a `def` function defined within the class.

```
class Account:
    def __init__(self):
        self._balance = 1000          # protected by convention: _name
        self.__pin = 9999             # private by name mangling: __name

    def get_balance(self):        # data is accessed via methods
        return self._balance

a = Account()
print(a.get_balance())     # 1000
# print(a.__pin) - AttributeError: private variable
```

Inheritance

- A child (derived) class reuses the attributes and methods of a parent (base) class.
- Types: **single**, **multilevel**, **hierarchical** and **multiple** inheritance.

```
class BaseTest:                       # parent
    def setup(self):
        print("launch browser")

class LoginTest(BaseTest):             # child - inherits setup()
    def test_login(self):
        print("login OK")

t = LoginTest(); t.setup(); t.test_login()

# Multiple inheritance - one child, two parents
class APIHelper:  pass
class DBHelper:   pass
class QAUtility(APIHelper, DBHelper):   # gets features of both
    pass
```

MRO — Method Resolution Order

What is MRO in Python? With (especially multiple) inheritance, MRO is the order in which Python searches classes for a method or attribute — left-to-right in the parent list, then upward. View it with `ClassName.mro()` or `__mro__`.

```
class A:
    def run(self): print("A")
class B(A):
    def run(self): print("B")
class C(A):
    def run(self): print("C")
class D(B, C):   # diamond: B first, then C, then A
    pass

D().run()            # B   - found in B first
print(D.mro())       # [D, B, C, A, object]
```

super() — call the parent

```
class BaseTest:
    def __init__(self, url):
        self.url = url

class LoginTest(BaseTest):
    def __init__(self, url, user):
        super().__init__(url)     # call parent constructor
        self.user = user
```

17. OOPs Part 3 — Polymorphism, Abstraction, Static Members

Polymorphism

Polymorphism = "many forms" — the same method name behaves differently depending on the object. The two flavours taught in the course:

- **Method Overriding** (the most common polymorphism in Python) — the child class *redefines* a method inherited from the parent.
- **Method Overloading** — same name, different parameters; Python emulates it with default arguments or `*args` (it does not support true overloading).

```
# 1) Method Overriding
class BaseTest:
    def run(self):
        print("run base suite")

class LoginTest(BaseTest):
    def run(self):                  # overrides parent's run()
        print("run login suite")

for t in [BaseTest(), LoginTest()]:
    t.run()                           # same call - different behaviour

# 2) Overloading emulated with default args
class Calc:
    def add(self, a, b, c=0):
        return a + b + c

calc = Calc()
print(calc.add(2, 3), calc.add(2, 3, 4))   # 5 9
```

Abstraction

- Abstraction in Python is a fundamental concept of OOP: it focuses on **hiding complex implementation details** while exposing only the essential features of an object.
- How to implement it:
  1. Import the `ABC` class and the `abstractmethod` decorator from the `abc` module.
  2. Define a class that inherits from `ABC`.
  3. Use the `@abstractmethod` decorator to define abstract methods (no body — subclasses must implement them).

```python
from abc import ABC, abstractmethod

class Report(ABC):                  # abstract class - cannot be instantiated
    @abstractmethod
    def generate(self):           # every child MUST implement this
        pass

class HTMLReport(Report):
    def generate(self):
        print("<html> report </html>")

class PDFReport(Report):
    def generate(self):
        print("PDF report")

for r in [HTMLReport(), PDFReport()]:
    r.generate()
```

Static method & class method

```python
class TestUtils:
    tool = "Selenium"                       # class (static) variable

    @staticmethod
    def is_valid_email(email):          # no self - acts like a plain function in the class
        return "@" in email

    @classmethod
    def with_tool(cls):                    # receives the class itself
        return f"running on {cls.tool}"

print(TestUtils.is_valid_email("a@b.com"))   # True - call without an object
print(TestUtils.with_tool())                  # running on Selenium
```

18. Exception Handling

What is an exception?

- Exceptions in Python are **events that occur during the execution of a program that disrupt the normal flow of instructions**.
- They are a way to handle **errors and unexpected situations gracefully** instead of crashing.
- Real scenarios where you will meet them: **dividing by zero, accessing an invalid list index, key missing in a dictionary, API connection failure, file not found**.

Types of exceptions

Exception	Raised when...
`NameError`	A local or global name is not found (undefined variable)
`TypeError`	An operation is performed on an incompatible data type
`ValueError`	A function receives an argument of the correct type but an inappropriate value
`IndexError`	Trying to access an index that doesn't exist in a sequence
`KeyError`	A key is not found in a dictionary
`ZeroDivisionError`	Dividing by zero

try / except / else / finally

```
try:
    a = int(input("num: "))
    result = 10 / a
except ValueError:
    print("Please enter a valid number")   # wrong value type
except ZeroDivisionError:
    print("Cannot divide by zero")         # a == 0
else:
    print(f"Result = {result}")              # runs only when NO exception occurred
finally:
    print("cleanup - always runs (close files, browser, DB)")
```

Catching the exception object + multiple exceptions + raise

```python
try:
    data = {"env": "qa"}
    print(data["url"])
except (KeyError, IndexError) as err:
    print(f"Missing key/index: {err}")

def set_age(age):
    if age < 0:
        raise ValueError("age cannot be negative")   # raise your own
    print("age saved")

set_age(-1)     # ValueError: age cannot be negative
```

QA pattern: wrap API calls in `try/except requests.ConnectionError` and file reads in `try/except FileNotFoundError` — never let a test suite crash mid-run.

19. Modules, Packages & PIP

Modules — import / export

- A module is simply a `.py` file with reusable code (functions, classes, variables).
- Types of modules:
  - **Built-in** — `os`, `math`, `re`, `sys`, `json`, `datetime`
  - **User-defined** — your own `.py` files
  - **External** — installed with pip: `requests`, `numpy`, `pandas`

```
# mymodule.py - export something
def greet(name):
    return f"Hello {name}"

# 170.py - import and use it
import mymodule
print(mymodule.greet("QA"))

from mymodule import greet          # import a single name
from mymodule import greet as g    # alias
import math as m                    # standard alias style
print(m.sqrt(16))                   # 4.0
```

Package vs directory

- A normal folder just groups files. When a folder contains an `__init__.py` file, **Python recognises it as a package** that can be imported.
- Course example: `package/util_module.py`, `package/util_module2.py`, `package/__init__.py` → `import package.util_module`.

```
package/
    __init__.py          # marks the folder as a package
    util_module.py       # helper functions (API helper, DB helper)
    util_module2.py

from package import util_module   # use the package
```

PIP — Python Installs Packages

- PIP is Python's package manager — it installs external libraries from PyPI.
- **Why pip matters** (the ecosystem this course relies on):
  - Web frameworks — Flask, Django
  - Data tools — Pandas, NumPy
  - AI libraries — Transformers, LangChain
  - Testing tools — PyTest, Playwright

- Automation tools — Requests, Selenium

```
pip --version                        # check pip
pip install requests                # install a package
pip install pytest==8.0.0            # specific version
pip install -r requirements.txt      # all project dependencies (pinned)
pip list                             # what is installed
pip show requests                    # details of one package
pip uninstall requests               # remove
```

Use cases for modules (from the course)

- Custom utilities — API helper, DB helper.
- **Page Object Model** separation for UI automation.
- Keeping data-driven logic in separate modules.

20. Virtual Environment (venv)

- A virtual environment (venv) **allows you to have multiple versions of Python / packages in a different folder** per project — so projects don't pollute each other.
- Used later in the course for DeepEval, CrewAI, LangChain, etc. (e.g. DeepEval works with Python >= 3.9 and recommends starting inside a venv).

```
# install virtualenv for your user (one time)
python -m pip install --user virtualenv

# create the environment in your working folder
virtualenv venv

# activate - Mac / Linux
source venv/bin/activate

# activate - Windows (PowerShell)
venv\Scripts\Activate.ps1

# now installs are isolated to the venv
pip install -U deepeval

# when finished
deactivate
```

Modern alternative used in the course: uv — `uv venv`, `uv pip install langflow`, `uv run langflow run` (docs.astral.sh/uv). Same concept, much faster.

21. OS Module (Operating System Interactions)

The `os` and `os.path` modules let Python talk to the operating system — essential for test automation:

- File handling & directory handling
- Environment variables
- Path manipulations
- Running shell commands
- Creating logs folders, reading `.env` configs, managing test data files, creating reports dynamically

```
import os

# --- current location ---
print(os.getcwd())                # current working directory

# --- directories ---
os.makedirs("reports/logs", exist_ok=True)   # create nested folders
print(os.listdir("."))             # list contents of a folder

# --- path manipulation (join, never hardcode separators) ---
report = os.path.join("reports", "logs", "run.txt")
print(os.path.exists(report))     # True / False

# --- environment variables ---
print(os.environ.get("PATH"))
os.environ["ENV"] = "qa"            # set one for this process

# --- running shell commands ---
os.system("echo hello from OS")
import subprocess
out = subprocess.run(["python", "--version"], capture_output=True, text=True)
print(out.stdout)
```

Never hardcode absolute paths (same rule the MCP module teaches): resolve paths relative to the file, e.g. `os.path.join(os.path.dirname(__file__), "td.csv")`, with an optional environment-variable override.

22. File IO & .env Config Files

Reading and writing files

Mode	Meaning
`"r"`	Read (default) — error if the file doesn't exist
`"w"`	Write — creates the file, **overwrites** existing content
`"a"`	Append — creates the file, adds to the end
`"rb"` / `"wb"`	Read / write in binary mode (images, PDFs)
`"r+"`	Read and write

```python
# write (course file: pramod.txt)
with open("pramod.txt", "w") as f:
    f.write("Line1\n")
    f.write("Line2\n")

# read - all at once
with open("pramod.txt", "r") as f:
    print(f.read())

# read - line by line (best for big files / logs)
with open("pramod.txt") as f:
    for line in f:
        print(line.strip())

# append
with open("pramod.txt", "a") as f:
    f.write("Line3\n")
```

> **Always use the `with` statement** — it closes the file automatically, even if an exception occurs inside the block.

Working with CSV test data (course file: td.csv)

```python
import csv
with open("td.csv") as f:
    rows = list(csv.DictReader(f))
    print(rows[0]["username"])   # each row is a dict - perfect for data-driven tests
```

.env config files (course file: .env.sample)

```
# .env  - never commit real secrets; keep a .env.sample with empty keys
BASE_URL=https://api.demoblaze.com
API_KEY=

# 176_Env.py - load it with python-dotenv
from dotenv import load_dotenv
import os

load_dotenv()                               # reads .env into environment
base_url = os.getenv("BASE_URL")       # read values
api_key  = os.getenv("API_KEY")
```

23. PyTest Basics for Test Automation

Why PyTest for this course: it lets you **group** tests, **manage** them, write **assertions** (check expected result vs actual result), validate, arrange test cases and **run them in parallel**.

Install and run

```
pip install pytest
pip install pytest-xdist          # parallel execution
pip install pytest-html          # HTML reports

pytest                           # run all tests
pytest -v                        # verbose - each test listed
pytest -s                        # show print() output
pytest test_180.py               # run one file
pytest -k login                  # run tests whose name contains 'login'
pytest -m smoke                  # run only tests marked 'smoke'
pytest -n 4                      # run in parallel on 4 workers (xdist)
```

Rules & first test

- Test files must be named `test_*.py` or `*_test.py`.
- Test functions must start with `test_`.
- Plain `assert` statements — no self.assertEqual needed.

```
# test_180.py
import pytest

def test_addition():
    assert 2 + 3 == 5           # expected == actual

def test_login(api_client):
    resp = api_client.post("/login")
    assert resp.status_code == 200
```

Expected vs actual — the core idea

```
def test_title():
    actual   = "3x AI Tester Blueprint"
    expected = "3x AI Tester Blueprint"
    assert actual == expected, f"expected {expected!r} but got {actual!r}"
```

Fixtures — setup / teardown (arrange your tests)

```python
import pytest

@pytest.fixture
def browser():
    print("launch browser")    # setup - before each test that asks for it
    yield "chrome"
    print("quit browser")     # teardown - after the test

def test_home(browser):      # fixture injected by name
    print(f"testing on {browser}")
```

Markers & parametrize (group and manage test cases)

```python
@pytest.mark.smoke               # group: run with  pytest -m smoke
def test_dashboard():
    assert True

@pytest.mark.parametrize("username,password", [
    ("admin", "admin123"),          # data-driven: one test - many rows
    ("guest", "guest123"),
])
def test_login(username, password):
    print(username, password)
```

Shared conftest.py

- Put common fixtures in `conftest.py` — PyTest picks them up automatically for every test file in that folder tree. No import needed.

24. Quick Revision Cheat Sheet

Concept	One-line memory hook
Python nature	High-level, human-friendly; compiled to .pyc then interpreted line by line
Keywords	Reserved words, case-sensitive, can never be identifiers
Identifier rules	letters/digits/underscore, no leading digit, case-sensitive
input()	Always returns a string — cast with int()/float()
Division	`/` float, `//` floor, `%` remainder, `**` power
String reverse	`s[::-1]` ; strings are immutable
range(1, 10, 2)	start included, stop excluded, step 2
do-while	No native support — emulate with `while True` + `break`
Switch	Python 3.10 `match/case` ; `case _:` = default
Function types	Built-in; user-defined: no-return, return, default param, keyword args, *args/**kwargs
Nested functions	Allowed in Python — foundation of decorators
Lambda	`lambda x: x * 2` — one-expression anonymous function
map vs filter	map transforms (same length), filter selects (True/False test)
Collection brackets	tuple `()` — list `[]` — set/dict `{}`
Set	Unordered, unique, unindexed; membership testing & de-dup
Class / Object	Class = blueprint; Object = run-time instance
Constructor	`__init__` , auto-called, no return; DC vs PC; `self` = current object
MRO	Method search order in inheritance; check with `Cls.mro()`
Polymorphism	Overriding = child redefines parent method; overloading = emulated via defaults
Abstraction	`abc.ABC` + `@abstractmethod` ; hide details, expose essentials
Exceptions	try/except/else/finally; Name/Type/Value/Index/Key/ZeroDivisionError
Package	Folder + `__init__.py` = importable package
PIP	Python Installs Packages: Flask, Django, Pandas, NumPy, LangChain, PyTest, Playwright, Requests, Selenium
venv	Project-isolated Python env; virtualenv / uv; venv\Scripts\activate (Win)
File modes	r read, w overwrite, a append; always use `with open(...)`
PyTest	test_*.py, test_ functions, plain assert, fixtures, markers, -n parallel

Next step in the blueprint: these basics feed directly into Chapter 12 (CrewAI), Chapters 17–18 (LangChain / LangGraph) and MCP creation with FastMCP — all Python-based AI-agent frameworks.